

Lab 1 Report

Display Density Metrics & Image Array Mechanics in NumPy

Human-Computer Interaction (HCI) & Computer Graphics (CG)

Student Name	Shahzad
Program	BSCS — University of Sindh
Roll No.	[2k24 / CSE / 139]
Submitted To	Sir Rajesh Kumar

Task 1 — Display Pixel Density (PPI/DPI) Calculator

A Python script prompts the user for horizontal resolution, vertical resolution, and physical diagonal screen size, then computes total pixel count, simplified aspect ratio (via `math.gcd`), diagonal pixel count, DPI/PPI, and a density classification.

```
"""
Lab 1 - Task 1: Display Pixel Density (PPI/DPI) Calculator
HCI & Computer Graphics
"""
import math

def classify_density(dpi):
    if dpi < 100:
        return "Low Density (Standard Monitor)"
    elif dpi <= 200:
        return "Medium Density (HD Display)"
    else:
        return "High Density (Retina / Mobile)"

def main():
    w_px = int(input("Enter horizontal resolution (pixels): "))
    h_px = int(input("Enter vertical resolution (pixels): "))
    d_inches = float(input("Enter physical diagonal size (inches): "))

    total_pixels = w_px * h_px

    g = math.gcd(w_px, h_px)
    aspect_w, aspect_h = w_px // g, h_px // g

    diagonal_px = math.sqrt(w_px ** 2 + h_px ** 2)
    dpi = diagonal_px / d_inches

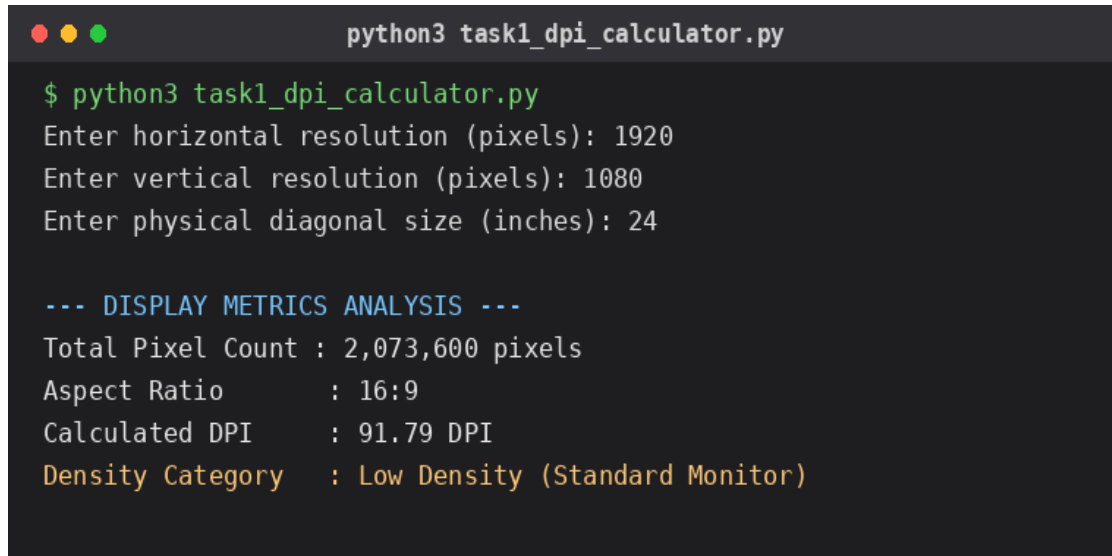
    category = classify_density(dpi)

    print("\n--- DISPLAY METRICS ANALYSIS ---")
    print(f"Total Pixel Count : {total_pixels:,} pixels")
    print(f"Aspect Ratio       : {aspect_w}:{aspect_h}")
```

```
print(f"Calculated DPI      : {dpi:.2f} DPI")
print(f"Density Category   : {category}")

if __name__ == "__main__":
    main()
```

Sample run 1 — Desktop monitor (1920 × 1080, 24"):



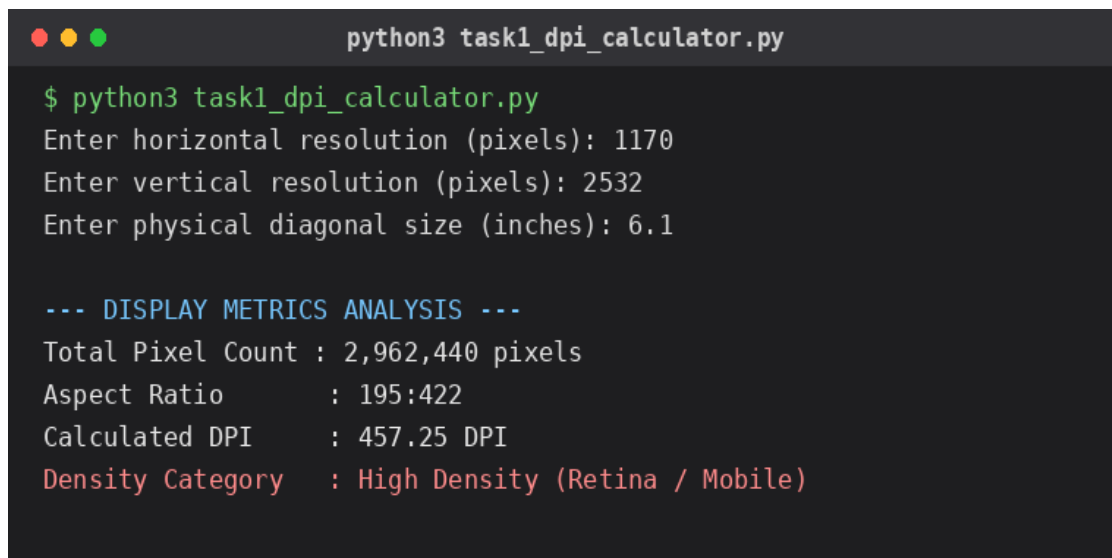
```
python3 task1_dpi_calculator.py

$ python3 task1_dpi_calculator.py
Enter horizontal resolution (pixels): 1920
Enter vertical resolution (pixels): 1080
Enter physical diagonal size (inches): 24

--- DISPLAY METRICS ANALYSIS ---
Total Pixel Count : 2,073,600 pixels
Aspect Ratio      : 16:9
Calculated DPI    : 91.79 DPI
Density Category  : Low Density (Standard Monitor)
```

Figure 1. Task 1 execution — desktop monitor.

Sample run 2 — Smartphone screen (1170 × 2532, 6.1"):



```
python3 task1_dpi_calculator.py

$ python3 task1_dpi_calculator.py
Enter horizontal resolution (pixels): 1170
Enter vertical resolution (pixels): 2532
Enter physical diagonal size (inches): 6.1

--- DISPLAY METRICS ANALYSIS ---
Total Pixel Count : 2,962,440 pixels
Aspect Ratio      : 195:422
Calculated DPI    : 457.25 DPI
Density Category  : High Density (Retina / Mobile)
```

Figure 2. Task 1 execution — smartphone screen.

Task 2 — Environment Setup & Synthetic Image Matrix Creation

A synthetic 300×400×3 uint8 array is created with `np.zeros()` and filled quadrant-by-quadrant using spatial slicing: top-left red, top-right green, bottom-left blue, bottom-right white.

```
"""
Lab 1 - Task 2: Environment Setup & Synthetic Image Matrix Creation
HCI & Computer Graphics
"""

import numpy as np
from PIL import Image

H, W, C = 300, 400, 3

img = np.zeros((H, W, C), dtype=np.uint8)

mid_h, mid_w = H // 2, W // 2

img[0:mid_h, 0:mid_w] = [255, 0, 0]          # Top-Left: Red
img[0:mid_h, mid_w:W] = [0, 255, 0]          # Top-Right: Green
img[mid_h:H, 0:mid_w] = [0, 0, 255]          # Bottom-Left: Blue
img[mid_h:H, mid_w:W] = [255, 255, 255]      # Bottom-Right: White

print("--- SYNTHETIC MATRIX METRICS ---")
print(f"Array Shape (H, W, C) : {img.shape}")
print(f>Data Type                : {img.dtype}")
print(f"Total Elements           : {img.size:,} values")
print(f"Memory Footprint          : {img.nbytes:,} bytes ({img.nbytes/1024:.2f} KB)")

Image.fromarray(img).save("synthetic_quadrants.png")
```

Console output:

```
--- SYNTHETIC MATRIX METRICS ---
Array Shape (H, W, C) : (300, 400, 3)
Data Type              : uint8
Total Elements         : 360,000 values
Memory Footprint       : 360,000 bytes (351.56 KB)
```

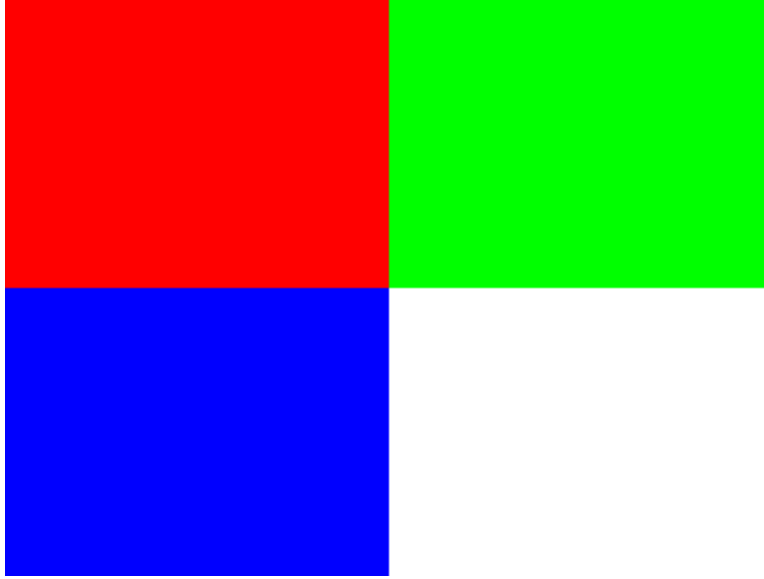


Figure 3. Synthetic 300×400 quadrant image (Red / Green / Blue / White).

Task 3 — Channel Slicing & Isolation

The sample image is loaded as a NumPy array and the Red, Green, and Blue channels are extracted via Axis-2 slicing. Each channel is also placed into an otherwise-zeroed 3-channel array to visualize it in color, and displayed alongside its grayscale intensity map in a 2×3 matplotlib subplot grid.

```
"""
Lab 1 - Task 3: Channel Slicing & Isolation
HCI & Computer Graphics
"""

import numpy as np
from PIL import Image
import matplotlib
matplotlib.use("Agg")
import matplotlib.pyplot as plt

img = np.array(Image.open("sample.jpg").convert("RGB"))

red_2d = img[:, :, 0]
green_2d = img[:, :, 1]
blue_2d = img[:, :, 2]

red_only = np.zeros_like(img)
red_only[:, :, 0] = red_2d

green_only = np.zeros_like(img)
green_only[:, :, 1] = green_2d

blue_only = np.zeros_like(img)
blue_only[:, :, 2] = blue_2d

print("--- CHANNEL EXTRACTION SUMMARY ---")
print(f"Original Image Shape : {img.shape}")
print(f"Red Channel 2D Shape : {red_2d.shape} | Mean Intensity: {red_2d.mean():.2f}")
print(f"Green Channel 2D Shape: {green_2d.shape} | Mean Intensity: {green_2d.mean():.2f}")
print(f"Blue Channel 2D Shape : {blue_2d.shape} | Mean Intensity: {blue_2d.mean():.2f}")
print("Display Window : Matplotlib 2x3 Subplot Grid Rendered.")

fig, axes = plt.subplots(2, 3, figsize=(13, 7))

axes[0, 0].imshow(red_only)
axes[0, 0].set_title("Red-Only")
axes[0, 1].imshow(green_only)
axes[0, 1].set_title("Green-Only")
axes[0, 2].imshow(blue_only)
axes[0, 2].set_title("Blue-Only")

axes[1, 0].imshow(red_2d, cmap="gray")
axes[1, 0].set_title("Red Channel (Grayscale)")
axes[1, 1].imshow(green_2d, cmap="gray")
axes[1, 1].set_title("Green Channel (Grayscale)")
axes[1, 2].imshow(blue_2d, cmap="gray")
axes[1, 2].set_title("Blue Channel (Grayscale)")

for ax in axes.flat:
    ax.axis("off")

fig.suptitle("Task 3: Channel Slicing & Isolation", fontsize=14)
```

```
fig.tight_layout()
fig.savefig("task3_channel_grid.png", dpi=130)
print("Saved figure: task3_channel_grid.png")
```

Console output:

```
--- CHANNEL EXTRACTION SUMMARY ---
Original Image Shape : (1080, 1920, 3)
Red Channel 2D Shape : (1080, 1920) | Mean Intensity: 144.09
Green Channel 2D Shape: (1080, 1920) | Mean Intensity: 160.44
Blue Channel 2D Shape : (1080, 1920) | Mean Intensity: 161.79
Display Window : Matplotlib 2x3 Subplot Grid Rendered.
```

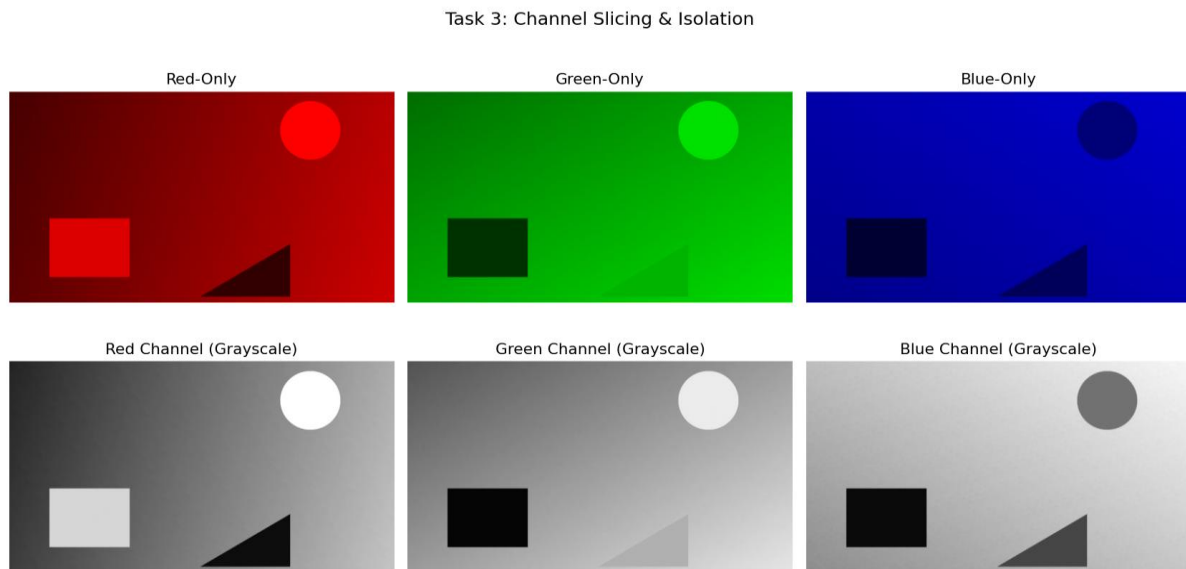


Figure 4. Top row: isolated color channels. Bottom row: grayscale intensity maps.

Task 4 — Spatial Downsampling & Pixelation via Striding

The image is downsampled by taking every 8th pixel along rows and columns (`img[::8, ::8, :]`), then re-expanded to the original resolution with `np.repeat()` to visualize the resulting pixelation. Dimension and memory reduction percentages are computed by comparing the original and downsampled array sizes.

```
"""
Lab 1 - Task 4: Spatial Downsampling & Pixelation via Striding
HCI & Computer Graphics
"""

import numpy as np
from PIL import Image
import matplotlib
matplotlib.use("Agg")
import matplotlib.pyplot as plt

N = 8

img = np.array(Image.open("sample.jpg").convert("RGB"))
orig_shape = img.shape
orig_mem = img.nbytes

downsampled = img[::N, ::N, :]
down_shape = downsampled.shape
down_mem = downsampled.nbytes

re_expanded = np.repeat(np.repeat(downsampled, N, axis=0), N, axis=1)
# trim/pad back to original shape in case of rounding mismatch
re_expanded = re_expanded[: orig_shape[0], : orig_shape[1], :]

dim_reduction_pct = (1 - down_shape[0] / orig_shape[0]) * 100
mem_savings_pct = (1 - down_mem / orig_mem) * 100

print(f"--- DOWNSAMPLING ANALYSIS (N = {N}) ---")
print(f"Original Shape      : {orig_shape} | Memory: {orig_mem:,} bytes")
print(f"Downsampled Shape   : {down_shape} | Memory: {down_mem:,} bytes")
print(f"Re-expanded Shape   : {re_expanded.shape} | Visual: Blocky Pixelation")
print(f"Dimension Reduction: {dim_reduction_pct:.2f}% reduction per axis")
print(f"Memory Savings      : {mem_savings_pct:.2f}% data reduction")

fig, axes = plt.subplots(1, 2, figsize=(12, 6))
axes[0].imshow(img)
axes[0].set_title(f"Original {orig_shape[1]}x{orig_shape[0]}")
axes[1].imshow(re_expanded)
axes[1].set_title(f"Re-expanded after N={N} Striding\n(Blocky Pixelation)")
for ax in axes:
    ax.axis("off")
fig.suptitle("Task 4: Spatial Downsampling & Pixelation", fontsize=14)
fig.tight_layout()
fig.savefig("task4_downsampling.png", dpi=130)
print("Saved figure: task4_downsampling.png")
```

Console output:

```
--- DOWNSAMPLING ANALYSIS (N = 8) ---
Original Shape      : (1080, 1920, 3) | Memory: 6,220,800 bytes
Downsampled Shape   : (135, 240, 3) | Memory: 97,200 bytes
Re-expanded Shape   : (1080, 1920, 3) | Visual: Blocky Pixelation
```

Dimension Reduction: 87.50% reduction per axis
Memory Savings : 98.44% data reduction

Task 4: Spatial Downsampling & Pixelation

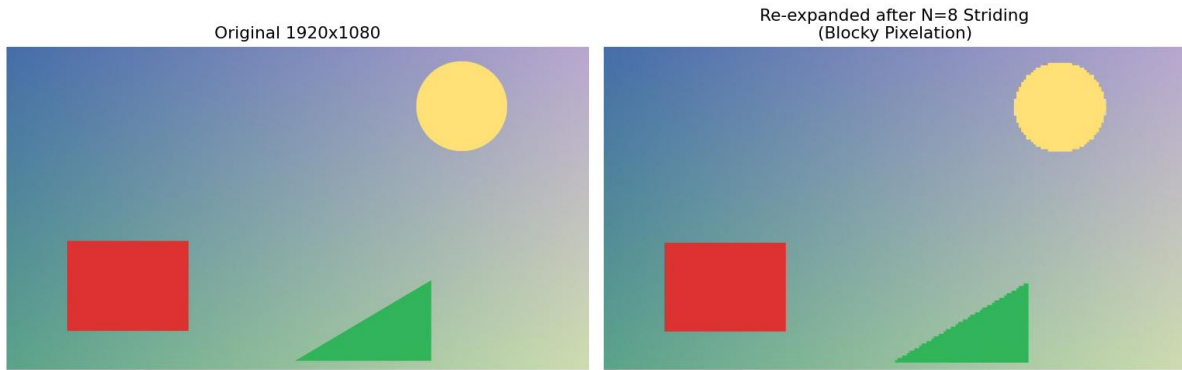


Figure 5. Original image (left) vs. re-expanded downsampled image (right), showing visible blocky pixelation.

Short Answer

Why does adding an Alpha channel (RGBA) increase an image array's memory consumption by 33% compared to standard RGB?

An image array's memory footprint is the product of its dimensions and the byte size of its data type: $nbytes = H \times W \times C \times (\text{bytes per element})$. For a standard uint8 image, each channel contributes exactly one byte per pixel, so memory scales linearly with the number of channels, C . Moving from RGB ($C = 3$: Red, Green, Blue) to RGBA ($C = 4$: Red, Green, Blue, Alpha) adds one extra byte per pixel for the alpha (transparency) channel, without changing H or W . The resulting memory ratio is $4/3 \approx 1.333$, i.e. a 33.3% increase — for example, a 1080×1920 RGB image uses 6,220,800 bytes, while the same image in RGBA uses 8,294,400 bytes, exactly 33.33% more. The increase is purely structural (one more equally-sized channel plane), not related to image content or compression.

Deliverables Checklist

- ✓ Documented Python source code (.py) for Tasks 1–4 — included above.
- ✓ Screenshots of Task 1 output for a desktop monitor (1920×1080, 24") and a smartphone (1170×2532, 6.1") — Figures 1–2.
- ✓ Matplotlib figure outputs for Task 3 and Task 4 — Figures 4–5.
- ✓ Short-answer response on RGBA memory overhead — above.